Technical Report — AUTOSAR HLD Analysis Assistant

Technical Report — AUTOSAR HLD Document Analysis Assistant

Academic Capstone Project Technical Report

Department of Computer Engineering / AI & ML

Pimpri Chinchwad College of Engineering (PCCOE), Pune

---

1. Title Page

• **Project Title**: AUTOSAR HLD Document Analysis Assistant

• **Candidate Details**: [Student Name(s)] | PRN: [Student PRN(s)] | CSx AIML

• **Project Mentor**: [Faculty Guide Name]

• **Academic Year**: 2025–2026 / 2026–2027

---

2. Abstract

Modern automotive software engineering relies on the AUTOSAR (AUTomotive Open System ARchitecture) standard to structure distributed electronic control unit (ECU) applications. High-Level Design (HLD) specifications define Software Components (SWCs), Port Interfaces, signal timing, and functional execution flows across hundreds of pages. Reviewing these documents manually is labor-intensive and prone to oversight. This project implements the **AUTOSAR HLD Document Analysis Assistant**, an end-to-end evidence-grounded AI assistant combining local dense embeddings, ChromaDB persistent vector indexing, and Google Gemini LLM reasoning. The system automatically extracts architectural entities, verifies cross-section consistency, highlights revision diffs, and answers natural-language queries with exact page-level citations. Tested across a 20-question automotive evaluation benchmark, the system achieved 100% Retrieval Recall@k, 100% Citation Validity, and 100% Abstention Accuracy on out-of-domain queries.

---

3. Introduction

Automotive software complexity has surged with the adoption of electrification, autonomous driving, and centralized domain controllers. High-Level Design (HLD) documents serve as the authoritative bridge between systems requirements and embedded implementation. However, their sheer size and intricate dependencies impede rapid onboarding, impact analysis, and peer review. This project delivers an automated assistant that extracts structured knowledge and validates architectural consistency with rigorous traceability.

---

4. Problem Statement

Automotive software engineers face significant bottlenecks in reviewing and maintaining AUTOSAR HLD documents:

1. Lengthy specifications (often 50–300+ pages) demand days of manual reading.

2. Inconsistencies between interface dictionaries and task scheduling sections frequently escape detection until integration testing.

3. Conventional PDF search is purely lexical and fails on semantic architectural queries.

4. Existing generic LLM tools hallucinate non-existent components, signals, and citations, which is unacceptable in safety-critical engineering.

---

5. Objectives & Scope

• **In-Scope**: Native PDF ingestion, table extraction, OCR fallback, local embedding generation, persistent ChromaDB indexing, grounded Q&A; with page citations, entity extraction (SWCs, Ports, Signals, Flows), revision diffing, rule-based inconsistency detection, empirical benchmarking.

• **Out-of-Scope**: Automated safety certification (ISO 26262), modifying source HLDs without review, or claiming full AUTOSAR standard compliance without official test suites.

---

6. Background & Related Work

The AUTOSAR Classic Platform Release 4.4.0 defines a layered model: Application Software Layer (ASW), Runtime Environment (RTE), and Basic Software (BSW). Previous work in requirements engineering utilized regex parsers or proprietary tools that lack natural-language query flexibility. Modern Retrieval-Augmented Generation (RAG) frameworks address this gap when augmented with strict grounding guardrails.

---

7. Requirements Analysis

• **Functional Requirements**:

• FR1: PDF text, table, and metadata extraction.

• FR2: Context-aware chunking preserving section titles and port definitions.

• FR3: Dense vector indexing and similarity retrieval.

• FR4: Grounded answers citing `[Doc: ..., Page: ..., Section: ...]`.

• FR5: Structured architectural entity extraction (Pydantic models).

• FR6: Revision comparison between document releases.

• FR7: Inconsistency detection for signal timing and undefined interfaces.

• **Non-Functional Requirements**:

• NFR1: Zero credential exposure in logs or UI.

• NFR2: Offline retrieval fallback if LLM credentials are absent.

• NFR3: Sub-second retrieval latency on standard PC hardware.

---

8. System Architecture & Component Decomposition

The application architecture is structured into modular layers:

1. **Ingestion & Preprocessing**: `src/ingestion.py`, `src/chunking.py`
2. **Representation & Vector Storage**: `src/embeddings.py`, `src/vector_store.py`
3. **Retrieval & Grounded Synthesis**: `src/rag_pipeline.py`
4. **Architectural Analysis**: `src/entity_extractor.py`, `src/doc_comparison.py`, `src/issue_detector.py`
5. **Presentation Layer**: Streamlit application (`app.py`)

---

9. Technology Stack & Model Configuration

• **Programming Language**: Python 3.13 / 3.11

• **UI**: Streamlit 1.65.0

• **Document Processing**: PyMuPDF 1.28.2, Pillow 11.0.0

• **Embeddings**: Sentence-Transformers `all-MiniLM-L6-v2` (384 dimensions)

• **Vector DB**: ChromaDB 1.5.9 (Local persistent storage)

• **LLM Engine**: Google Gemini API via official `google-genai` SDK

• **Testing**: pytest 9.1.1 (23 test cases)

---

10. Document Ingestion & OCR Methodology

PyMuPDF sequentially extracts text blocks while capturing font headers and markdown tables. Pages containing fewer than 30 characters trigger an automated OCR check. If Tesseract is present on the host system, page pixmaps are rasterized and processed via `pytesseract`. If Tesseract is absent, the system logs an informative message and proceeds in native extraction mode.

---

11. Embedding & Vector Database Design

Every extracted text chunk is transformed into a 384-dimensional dense vector using `all-MiniLM-L6-v2`. Vectors and primitive metadata dictionaries (`doc_id`, `filename`, `page_num`, `section`, `chunk_index`, `doc_hash`, `char_count`) are stored in ChromaDB using cosine distance. Duplicate indexing is blocked by comparing SHA-256 document checksums.

---

12. RAG Pipeline & Grounding Guardrails

When a user submits a query:

1. The query vector is computed and top-k (k=4) chunks are retrieved.

2. If query terms are completely absent from document vocabulary, the pipeline immediately triggers grounded abstention.

3. If relevant passages exist, they are compiled into a formatted prompt enforcing strict system constraints: no hallucination, mandatory page citations, and explicit uncertainty reporting.

4. If Gemini is unavailable, the pipeline operates in **Retrieval-Only Mode**, displaying retrieved evidence excerpts with exact citations.

---

13. Citation & Grounding Design

Citations follow the syntax: `[Doc: <filename>, Page: <page>, Section: <section>]`. The `_validate_citations()` module validates emitted citations against the retrieved candidate set, verifying that the cited page and document actually contain supporting evidence.

---

14. Architectural Entity Extraction

Using domain regex rules and Pydantic validation schemas, the extractor categorizes:

• **Components**: SWCs (e.g., `ThrottleControl_SWC`, `BrakeManager_SWC`)

• **Ports**: Provided (`Pp...`) and Required (`Rp...`) Ports

• **Interfaces**: Sender-Receiver interfaces (`If_...`)

• **Signals**: Data elements with types, units, and cycle times (`Sig_...`)

• **Functional Flows**: End-to-end execution sequences

---

15. Document Revision Comparison & Inconsistency Detection

The `DocumentComparator` matches sections across revisions, highlighting additions (e.g. Cybersecurity Section 7 in v2.0), modifications, and entity deprecations.

The `IssueDetector` executes rule-based inconsistency checks:

• **Conflicting Definitions**: Different update rates for the same signal (e.g. 10ms vs 50ms).

• **Undefined Interfaces**: Ports citing interfaces omitted from the catalog.

• **Undeclared Components**: SWCs referenced in functional flows but missing from Section 3.

• **Broken Cross-References**: References to non-existent sections.

---

16. Security & Data Privacy

• Secrets are managed strictly via `.env` and `python-dotenv`.

• API keys are never printed in logs, UI, exports, or error tracebacks.

• Retrieval-only mode operates 100% locally on the host workstation.

---

17. Automated Testing Methodology

A 23-test automated suite (`pytest tests/ -v`) validates:

• Checksumming and text normalization

• Section and table chunking

• Duplicate indexing prevention and safe deletion

• RAG retrieval, citations, and abstention

• Entity extraction and revision comparison

• CSV and JSON serialization

• Controlled mock tests of Gemini LLM integration

Result: 23 passed, 0 failed (100% pass rate).

---

18. Evaluation Dataset & Benchmark Setup

The system was evaluated using `Input_Data/evaluation_dataset.json`, comprising 20 curated questions covering:

• Factual component lookups

• Signal dictionary specifications

• Multi-step functional flows

• Cross-section inconsistency detection

• Deliberately unsupported negative questions

---

19. Empirical Evaluation Results

---

20. Known Limitations

1. Does not reconstruct semantic wiring graphs from graphical vector diagrams without multimodal models.

2. Requires host Tesseract binary for scanned pages.

3. Requires internet connectivity and API quota for live Gemini conversational generation.

4. Does not constitute certified ISO 26262 ASIL approval.

---

21. Future Enhancements

• Integration with multimodal vision models (e.g. Gemini Multimodal) for block diagram wire tracing.

• Support for AUTOSAR ARXML machine-readable XML format ingestion.

• Automated generation of AUTOSAR ECU Configuration (ECUC) templates.

---

22. Conclusion

The **AUTOSAR HLD Document Analysis Assistant** delivers a demonstrable, rigorously tested, and evidence-grounded tool that significantly reduces manual engineering effort in reviewing complex automotive architectures while ensuring full citation traceability.

---

23. References

1. AUTOSAR Consortium, "Explanation of Application Interfaces of the Powertrain Domain", Release 4.4.0.

2. ISO 26262:2018, "Road vehicles — Functional safety".

3. Lewis, P., et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS 2020.

4. Google Cloud, "Google Gen AI Python SDK Documentation", 2025.